Part 2: Design Models - Equilibrium & Linearization

This section covers finding the equilibrium point of the nonlinear system and deriving a linear model for control design.

Python Scripts Needed

- `practiceFinalSim2.py`: The main simulation script for Part 2.
- `design_model_tool.py`: Numerical tool for finding equilibrium and linearization matrices.
- `eq_controller.py`: Implementation of the feedforward equilibrium control.
- `params.py`: Linearization results (A, B, C, D) and transfer function $(P(s))$.

2.1 - 2.2: Equilibrium Analysis

Description

Determine the control input (torque) required to hold the system at a specific angle.

What to do:

1. **Analytical Method**: Set $\ddot{\theta} = 0, \dot{\theta} = 0$ in the EOM and solve for τ_e.

$$\tau_e = mg\ell \cos\theta_e + k_1\theta_e + k_2\theta_e^3$$

2. **Numerical Method**: In `design_model_tool.py`, the `find_equilibrium` method computes this for you. Verify it matches your calculation.
3. **Action**: Record τ_e for $\theta_e = 0°$ and $\theta_e = 30°$.

2.3 - 2.5: Linearization & State-Space

Description

Create a linear approximation of the system near the equilibrium point.

What to do:

1. **Symbolic Partial Derivatives**: Calculate $\frac{\partial f}{\partial \theta}$, $\frac{\partial f}{\partial \dot{\theta}}$, and $\frac{\partial f}{\partial \tau}$ at $(0, 0, \tau_e)$.
2. **Formulate A, B, C, D**:

$$A = \begin{bmatrix} 0 & 1 \\ \frac{\partial f}{\partial \theta} & \frac{\partial f}{\partial \dot{\theta}} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{\partial f}{\partial \tau} \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

3. **Numerical Linearization**: Run `python design_model_tool.py` to get the numerical A, B, C, D matrices.
4. **Transfer Function**: The tool also provides $P(s) = C(sI - A)^{-1}B + D$. Record this for PID tuning.

2.6: Simulation Verification

Description

Verify that the equilibrium torque u_e actually holds the nonlinear system at the desired point.

What to do:

1. **Update `params.py`**: Set `u_e` to your calculated value.
2. **Run Simulation**: Execute `python practiceFinalSim2.py`.
3. **Observation**: The system should stay perfectly still at the equilibrium angle with zero initial error.